



GALERIA DE FOTOS

INTEGRANTES

- Erazo Salvatierra, Danitza Anahi (Código: 26102434)
- Chavez Rivera, Yeongmi Sayuri Milagros (Código: 26100408)
- Rojas Coronel, Nayeli Madelein (Código: 26102022)
- Monteverde Maravi, Celery Lindsay (Código: 26101240)

Sistemas Operativos I - Proyecto del Curso



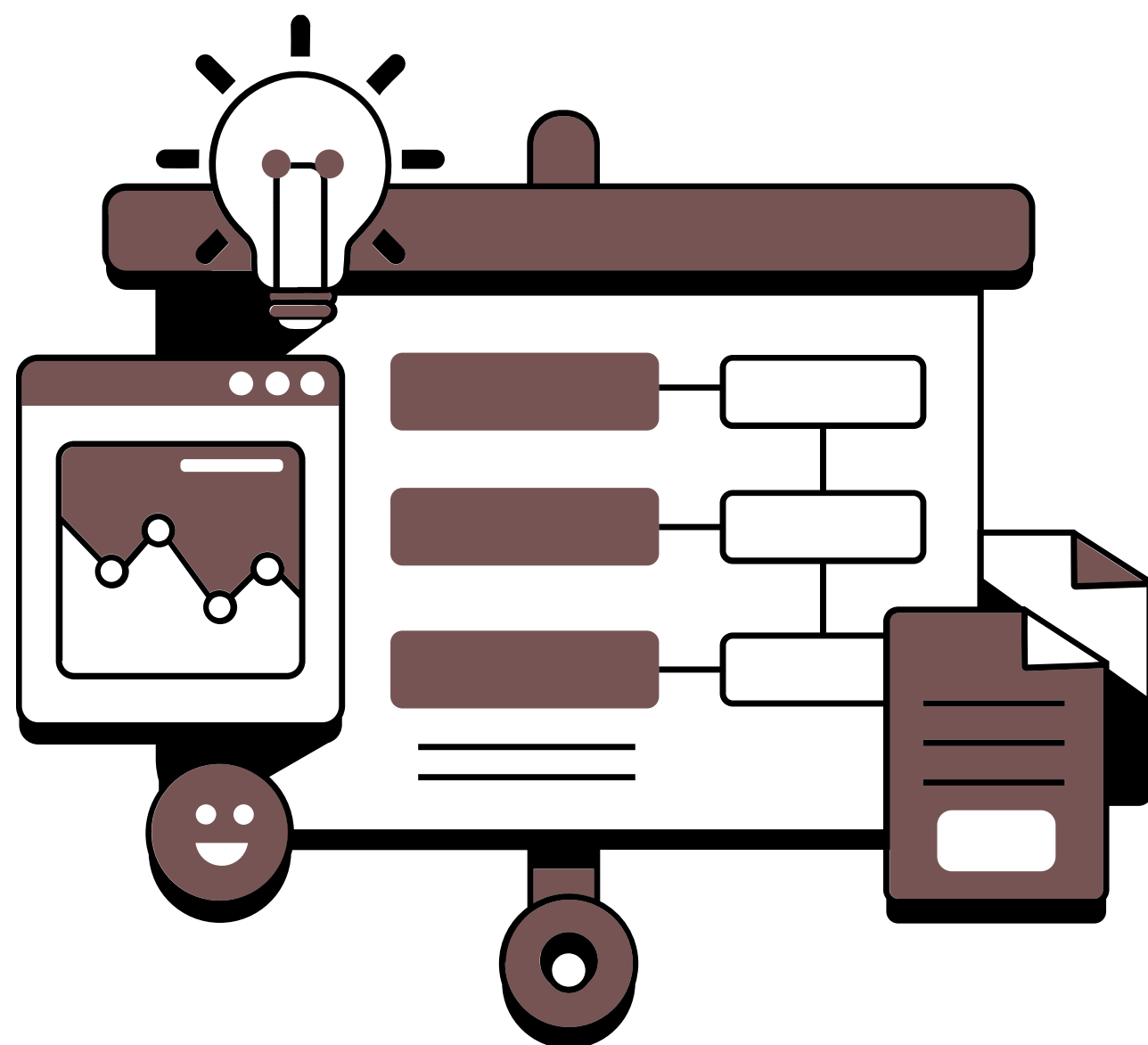
UNIVERSIDAD
esan



UNIVERSIDAD
esan

IMMICH

Galería de Fotos con IA Auto-hospedada



¿QUE ES IMMICH?

Plataforma de código abierto para gestión de fotografías y videos.

¿PARA QUÉ SIRVE?

- Almacenar y organizar fotos y videos de manera privada
- Acceso desde cualquier dispositivo (web y móvil)
- Sincronización automática
- Búsqueda inteligente por rostros, objetos y lugares

¿QUÉ LA HACE ATRACTIVA?

- Alternativa open source a Google Photos
 - Privacidad total: tus datos son tuyos
 - Motor de IA integrado (Machine Learning)
 - Escalable y personalizable
-

ARQUITECTURA DEL DESPLIEGUE

```
INTERNET (Usuario)
|
galeria-sistemas-operativos.duckdns.org (DuckDNS)
| IP Elástica: 3.230.136.170
+-----+-----+ Security Group AWS
| EC2 t3.micro - Ubuntu 24.04 LTS |
| Puerto 80 (HTTP) -> immich_server |
| Red interna: immich_default (bridge) |
| [immich_server:2283] |
| [immich_machine_learning] (lim 50MB) |
| [immich_postgres] (pgvector) |
| [immich_redis] (Valkey/Redis) |
| Volumen: ./immich-data (fotos+DB) |
+-----+-----+
```

CADA SERVICIO SE EJECUTA DENTRO DE UN CONTENEDOR PODMAN EN MODO ROOTLESS. EL USUARIO 'UBUNTU' ES PROPIETARIO DE TODOS LOS PROCESOS.

Entorno de despliegue:

- Plataforma: AWS EC2 (tipo t3.micro, 1 GB de RAM física)
- Sistema Operativo: Ubuntu (última versión LTS)
- Almacenamiento inicial: 30 GB EBS (máximo capa gratuita)

ARQUITECTURA DEL DESPLIEGUE

CONTENEDOR	FUNCIÓN	PUERTO
IMMICH_SERVER	SERVIDOR WEB (NESTJS)	80 (HOST) : 2283
IMMICH_POSTGRES	BASE DE DATOS POSTGRESQL	INTERNO
IMMICH_REDIS	CACHÉ (VALKEY/REDIS)	INTERNO
IMMICH_MACHINE_LEARNING	MOTOR DE IA (PYTORCH)	INTERNO

VOLÚMENES PERSISTENTES:

- ./immich-data → Fotos originales, thumbnails y videos.
- ./postgres → Base de datos vectorial (PostgreSQL + pgvector).

LÍMITE DE RECURSOS (Cgroups):

- immich_machine_learning: 50 MB (límite efectivo ~31 MB)



UNIVERSIDAD
esan

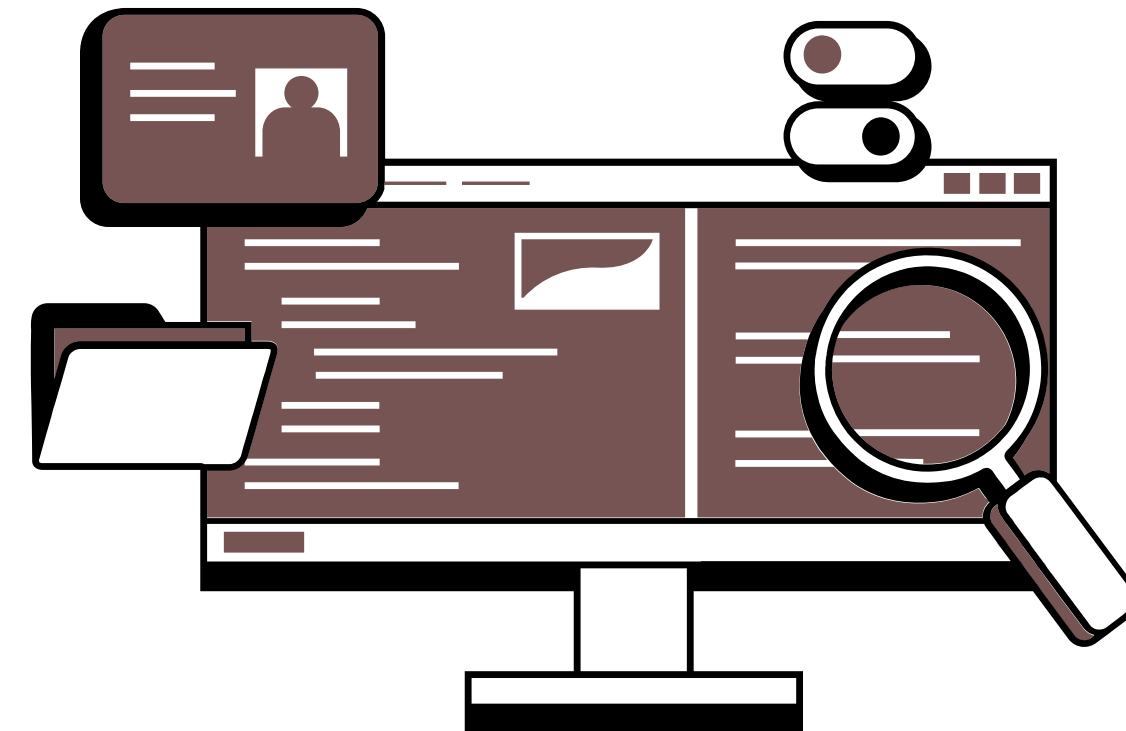
ARQUITECTURA DEL DESPLIEGUE

VOLÚMENES PERSISTENTES:

- /data → Fotos originales y miniaturas
- model-cache → Modelos de IA

¿POR QUÉ PODMAN ROOTLESS?

- Cada contenedor es un proceso hijo del usuario
- No requiere demonio con privilegios root
- Podemos inspeccionar con ps aux, /proc, cgroups
- Seguridad: menor superficie de ataque



"PODMAN NO ESCONDE LOS MECANISMOS DEL KERNEL: LOS DEJA A LA VISTA PARA ESTUDIARLOS."

MEMORIA VIRTUAL Y CONTROL DE RECURSOS CON CGROUPS

TEORÍA APLICADA:

• MEMORIA PRINCIPAL (RAM)

- Memoria volátil donde se ejecutan los procesos
- Recursos limitados → Hay que administrarlos

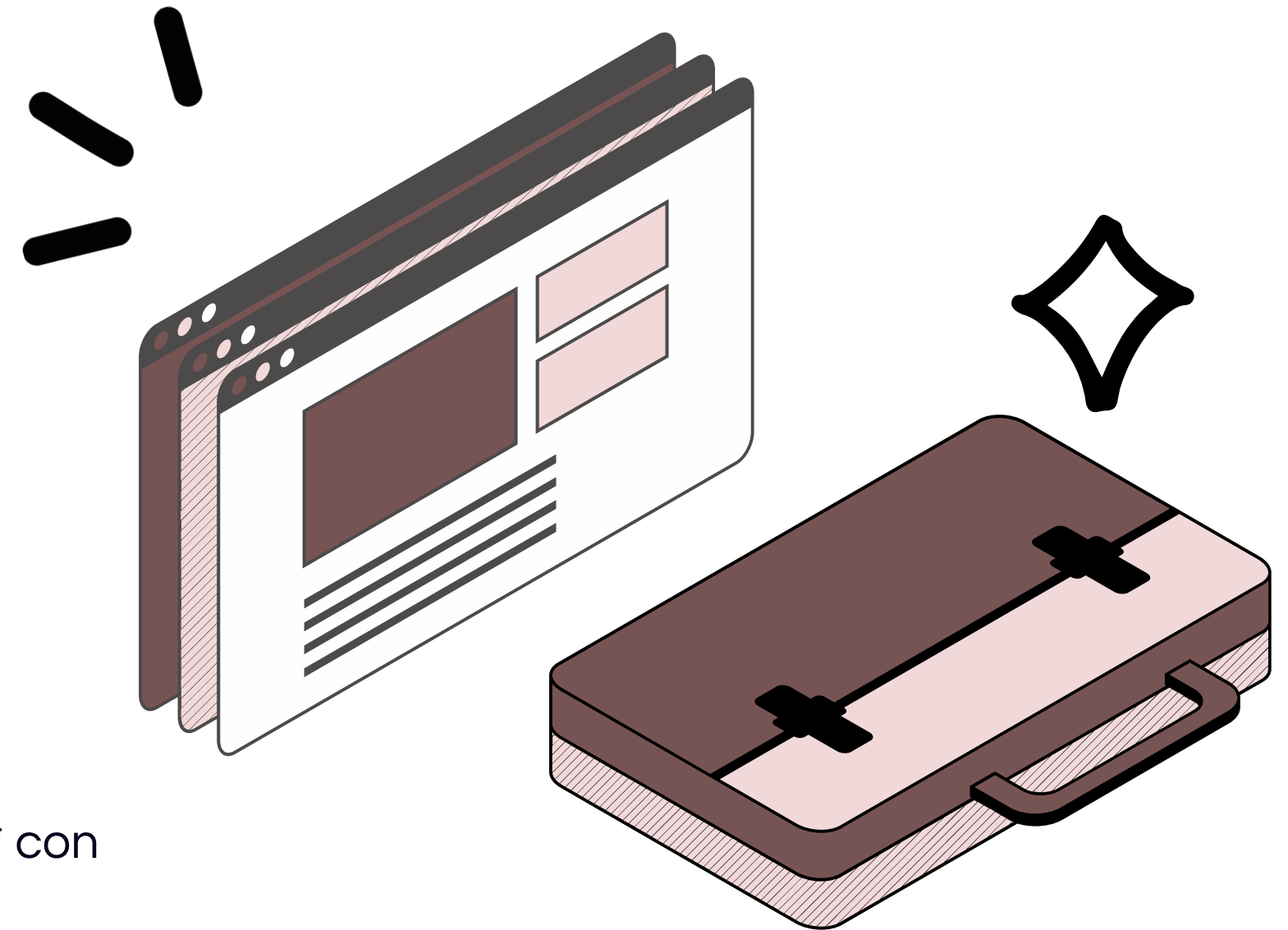
• MEMORIA VIRTUAL

- Extensión de la RAM usando disco (Swap)
- Permite ejecutar procesos más grandes que la RAM física
- Fallos de página: cuando un dato no está en RAM

• CGROUPS (CONTROL GROUPS)

El contenedor de Machine Learning (IA) es *demandante de RAM* con 50 MB es insuficiente

- Aplicamos límite de *50 MB MB* usando cgroups
- Demostramos cómo el SO controla el consumo



Comandos Clave de Memoria Virtual

FREE -H

Memoria RAM y Swap disponible

CAT /PROC/MEMINFO

Información detallada de memoria

VMSTAT

Actividad de memoria, procesos y CPU

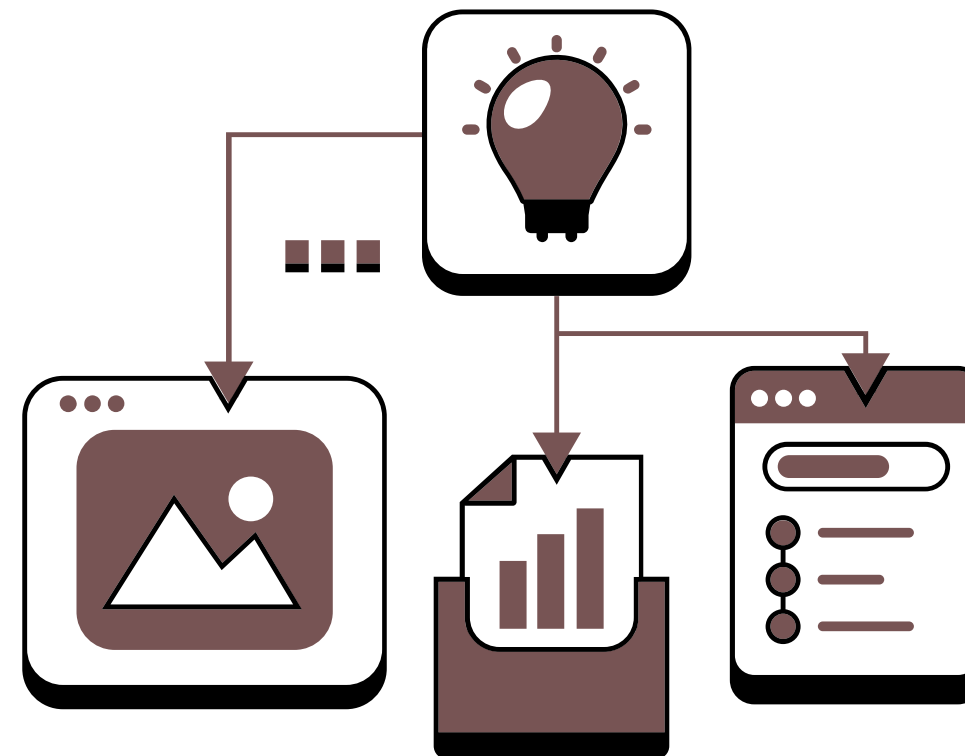
PMAP [PID]

Mapa de memoria de un proceso

PODMAN STATS

Consumo de recursos de contenedores

COMANDOS



COMANDOS ADICIONALES (TROUBLESHOOTING)

PODMAN LOGS IMMICH_SERVER

Ver errores del servidor web

PODMAN LOGS IMMICH_POSTGRES

Ver errores de la base de datos

PODMAN INSPECT IMMICH_POSTGRES

Ver configuración y red

SWAPON --SHOW

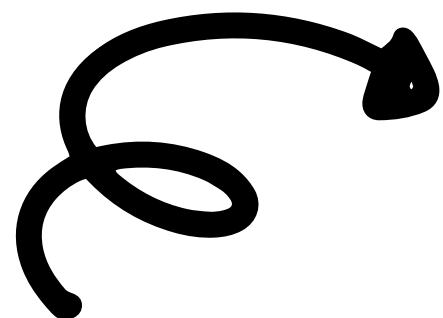
Ver memoria Swap activa

CD PROYECTO-SISTEMAS

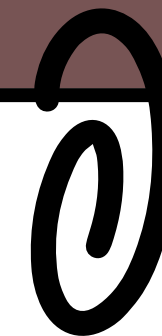
Navegar al directorio del proyecto

DF -H

Ver espacio en disco



DEMO EN VIVO-IMMICH FUNCIONANDO



1. VERIFICACIÓN DEL ENTORNO:

1. **\$ CD PROYECTO-SISTEMAS Y \$ PODMAN PS :**
2. **\$ CURL -I HTTP://LOCALHOST:**

2. ACCESO A LA GALERÍA

1. **ABRIR NAVEGADOR Y PONER**
HTTP://GALERIA-SISTEMAS-
OPERATIVOS.DUCKDNS.ORG
2. **INICIAR SESIÓN EN IMMICH.**

3. CARGA DE IMÁGENES (PROCESAMIENTO)

1. Subir fotografías desde la interfaz web.
2. El servidor encola las tareas para el Motor de IA.

4. MONITOREO EN TIEMPO REAL (CONSUMO DE RAM)

\$ PODMAN STATS --NO-STREAM

Nos da como Resultado:

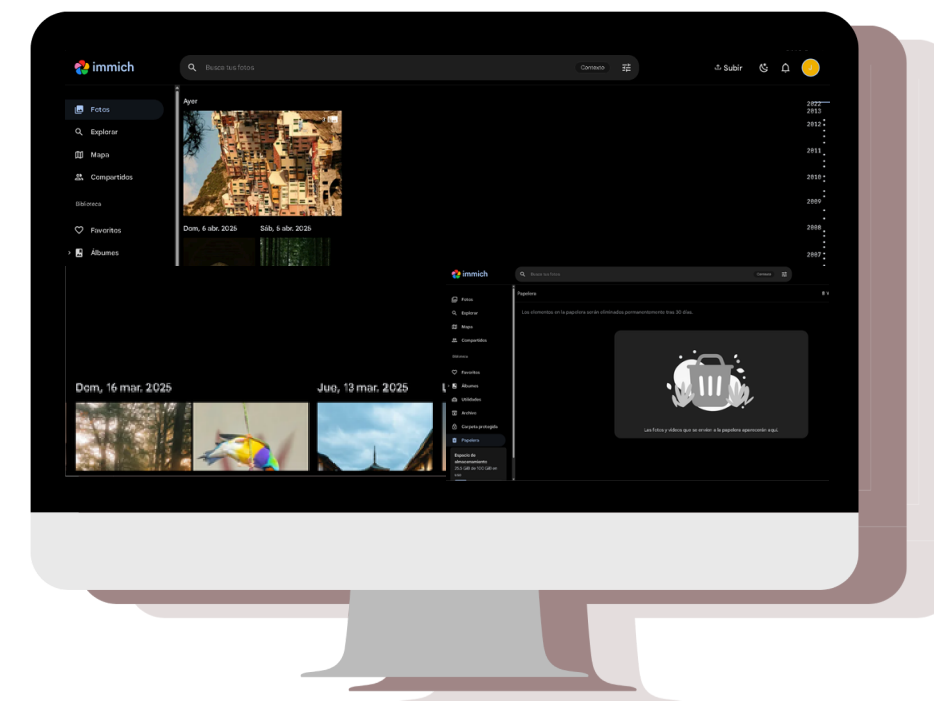
immich_machine_learning usa ~29 MB /
31.46 MB (94.71% del límite).

5. VERIFICACIÓN DEL OOM KILLER (EFECTO DEL LÍMITE)

Se ejecuta en la terminal

\$ PODMAN PS -A

Nos da como Resultado:
immich_machine_learning aparece con
"Up 8 seconds" (reinicio constante),
mientras los otros llevan "Up 45 minutes".



EL SO POR DEBAJO DEL CONTENEDOR

1. VERIFICACIÓN DEL LÍMITE EN CGROUPS V2

- Obtener el PID del contenedor:

```
$ podman inspect immich_machine_learning | grep -i cgroup
```

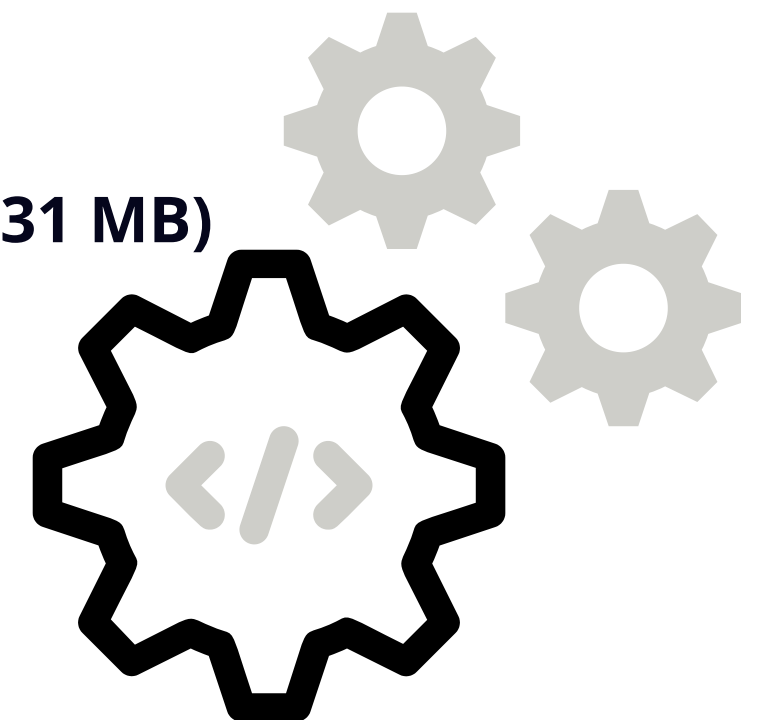
- Leer el límite impuesto por el kernel:

```
$ cat /sys/fs/cgroup/.../memory.max: 52428800 bytes (~50 MB configurados, efectivo ~31 MB)
```

2. MONITOREO EN TIEMPO REAL

```
$ podman stats --no-stream
```

CONTAINER	ID NAME	CPU %	MEM USAGE / LIMIT	MEM %
ABC123DEF456	IMMICH_MACHINE_LEARNING	45.2%	242.3MIB / 250MIB	96.9%



3. COMPROBACIÓN DESDE EL HOST

```
$ ps aux | grep immich
```

- El proceso Python del ML consume ~30 MB, exactamente dentro del límite asignado.

EL KERNEL DE LINUX (CGROUPS) APLICA EL LÍMITE DE 250 MB AL CONTENEDOR DE IA, PROTEGIENDO AL SISTEMA ANFITRIÓN.

EXPERIMENTO: IMPACTO DEL LÍMITE DE MEMORIA (OOM KILLER)



```
ubuntu@ip-172-31-19-175:~/proyecto-immich$ sudo podman ps -a
CONTAINER ID   IMAGE                                     COMMAND                  CREATED        STATUS
PORTS         NAMES
2b5fce5e8a4e   docker.io/library/ubuntu:latest         /bin/bash -c x=a;...    21 hours ago   Exited (2) 21 ho
urs ago       victima_oom
8cadb2da696c   ghcr.io/immich-app/immich-machine-learning:v2  python -m immich_...    16 seconds ago Up 16 seconds
immich_machine_learning
8d64bfb648a9   docker.io/valkey/valkey@sha256:3b55fbaa0cd93cf0d9d961f405e4dfcc70efe325e2d84da207a0a8e6d8fde4f9  valkey-server          15 seconds ago Up 15 seconds (s
tarting)      immich_redis
eb556cecb400   ghcr.io/immich-app/postgres@sha256:bcf63357191b76a916ae5eb93464d65c07511da41e3bf7a8416db519b40b1c23  postgres -c confi...   13 seconds ago Up 13 seconds
immich_postgres
7241495c7de1   ghcr.io/immich-app/immich-server:v2      start.sh                12 seconds ago Up 12 seconds
0.0.0.0:80->2283/tcp  immich server
```

ESCENARIO	CONFIGURACIÓN	RESULTADO
A) Sin Swap	1 GB RAM real, sin swap, sin límite en ML.	El sistema se congela y colapsa al iniciar los 4 contenedores. SSH se desconecta.
B) Con Swap + Límite de 50 MB	1 GB RAM + 4 GB Swap, límite estricto de 50 MB en el ML (cgroups).	El sistema sobrevive. El OOM Killer mata el ML (SIGKILL, código 137), pero el servidor web, Redis y PostgreSQL siguen operativos.

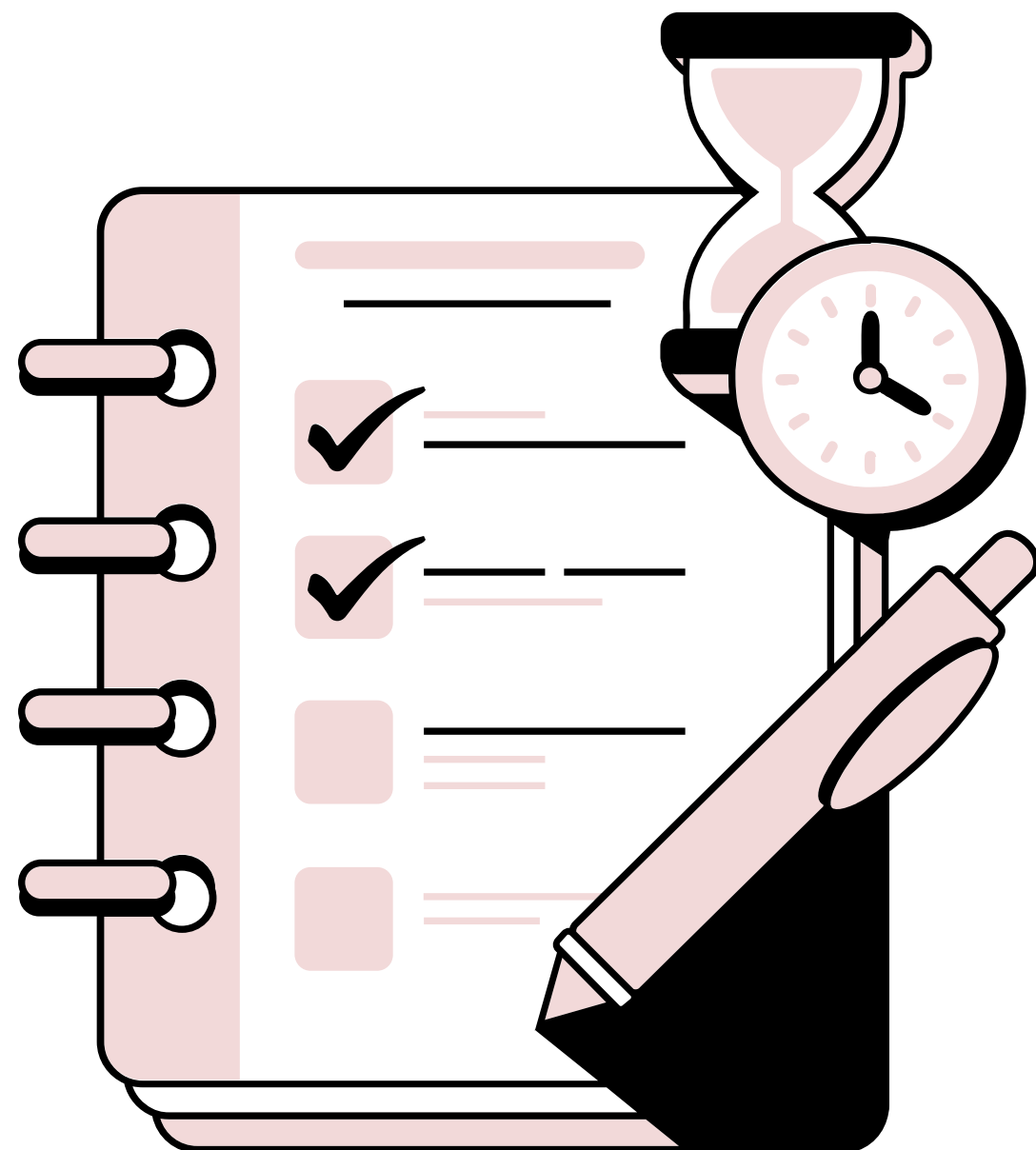
SOLUCION:

- sudo fallocate -l 4g/swapfile
- sudo mkswap /swapfile
- sudo swapon /swapfile

CONCLUSIÓN:

El límite de cgroups y la memoria swap permiten aislar procesos problemáticos sin comprometer la estabilidad del sistema anfitrión.

INTERPRETACION DE RESULTADOS



ASPECTO	T1 (INICIO)	T2 (POST-INDEXACIÓN)	VARIACIÓN
immich_server	273.9 MB	371.8 MB	+35%
immich_machine_learning	22.45 MB	22.45 MB	Estable (limitado)
immich_postgres	5.7 MB	13.74 MB	+138% (vectores)
immich_redis	2.9 MB	3.7 MB	Mínimo

EFECTO DEL OOM KILLER:

El kernel mata el ML con SIGKILL, pero los demás contenedores permanecen estables gracias al aislamiento de cgroups.

CONCLUSION

1. APRENDIZAJES TÉCNICOS:

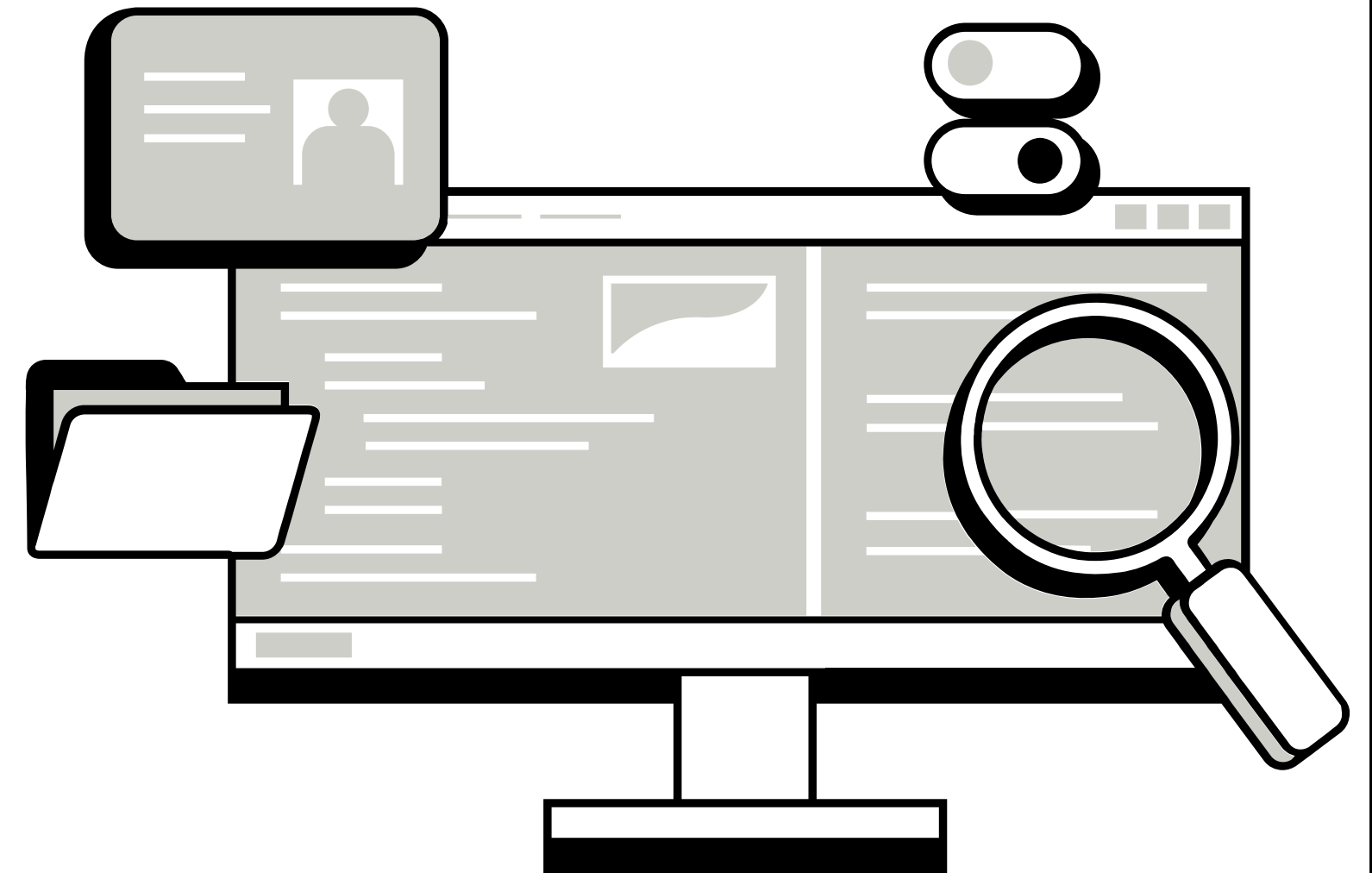
- Resolver problemas de red en Podman (CNI, DNS interno)
- Inspeccionar contenedores con podman inspect
- Aplicar límites con Cgroups y verificarlos en /sys/fs/cgroup
- Monitorear en tiempo real con podman stats

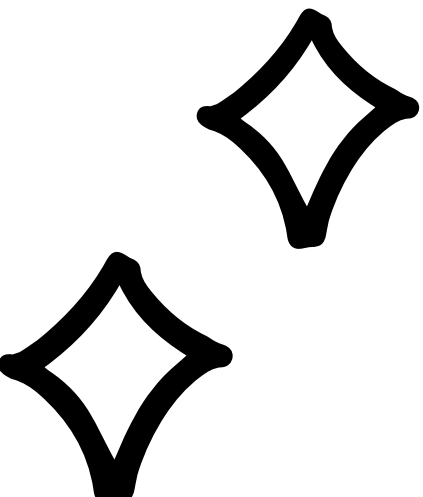
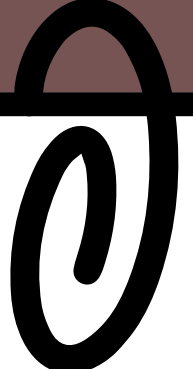
2. LIMITACIONES DEL SISTEMA:

- Instancia t3.micro con 1 GB RAM real + 4 GB swap (5 GB de espacio de direcciones virtual)
- Sin GPU → procesamiento de IA más lento
- Latencia en lotes grandes de imágenes

3. MEJORAS FUTURAS:

- Aumentar límite de 512 MB o 1 GB en producción
- Integrar GPU para acelerar ML
- Implementar Prometheus + Grafana para monitoreo
- Autoinicio con podman generate systemd





**MUCHAS
GRACIAS**

